**ISLAMIC UNIVERSITY OF TECHNOLOGY**

**DHAKA, BANGLADESH**

**ORGANIZATION OF ISLAMIC COOPERATION**

**DATABASE MANAGEMENT SYSTEMS LAB**

**Name: Antara Arifa Mullick**

**Student ID: 210042132**

**Department: Computer Science and Engineering**

**Program: B.Sc. in Software Engineering**

## Report preparing for Lab Task

[A - Back to Underworld]

In this task, we are presented with a scenario of a war between Vampires and Lykans. Humans want to determine the maximum possible number of members from either race. The information is collected from dual fights, and the goal is to find the count.

### Code Snippet

```
#include <bits/stdc++.h>
using namespace std;

vector<int> vec[20009];
bool vis[20009]= {};
bool node_type[20009]= {};

long long int bfs(int x);

void clr()
{
    memset(vis,false,sizeof (vis));
    memset(node_type,false,sizeof (node_type));
    for(int i=0; i<20009; i++)
        vec[i].clear();
}

long long int modified_bfs(int x);

int main()
{
    int T,u,v,n,case_num=1;
    scanf("%d",&T);

    while(T--)
    {
        clr();

        scanf("%d",&n);
        long long int sum=0;

        while(n--)
        {
            scanf("%d %d",&u,&v);
            vec[u].push_back(v);
```

```
            vec[v].push_back(u);
        }

        for(int i=0; i<20009; i++)
        {
            if(vis[i]==0 && !vec[i].empty())
            {
                sum+= modified_bfs(i);
            }
        }
        cout<<"Case "<<case_num++<<": "<<sum<<'\n';
    }
    return 0;
}
long long int modified_bfs(int x)
{
    long long int lykan_num=0,vampire_num=0;
    queue<int>q;

    q.push(x);
    vis[x]=1;
    node_type[x]=1;
    ++lykan_num;

    while(!q.empty())
    {
        int u = q.front();
        q.pop();

        for(int i=0; i<vec[u].size(); i++)
        {
            int v = vec[u][i];

            if(vis[v]==0)
            {
                q.push(v);
                vis[v]=1;

                if(node_type[u]==1)
                {
                    node_type[v] = 0;
                    ++vampire_num;
                }
                else
                {
                    node_type[v] = 1;
                    ++lykan_num;
                }
            }
        }
    }

    long long int mx = max(vampire_num,lykan_num);

    return mx;
}
```

**Code Explanation**

Initialization: The code initializes necessary data structures, an array of vectors to represent dual fights and arrays to track visited nodes and their types (Vampire or Lykan).

Modified BFS Function: The core of the solution is the modified_bfs function. Starting from a specific node (x), it performs a DFS traversal.

DFS Exploration: It explores the dual fights, marking visited nodes and finding the number of Vampires and Lykans.

BFS Queue: The traversal is managed by a queue (q), which starts at the first node (x).

Node Type and Counts: When a node is visited, the code counts the number of vampire or lykan nodes it has encountered.

Maximum Count: The function gives the highest number of Lykans or vampires that can be found within the group.

Main Function: The programme processes each test case, reads in the number of test cases (T), and prints the results for as many members as possible.

## B - Kefa and Park

In this task, Kefa needs to choose a restaurant in a park with a tree structure. However, he is afraid of cats and wants to find restaurants that are not too close to consecutive cat locations. The problem involves identifying suitable restaurants.

**Code Snippet**

```cpp
#include <iostream>
#include <vector>

using namespace std;

const int MAX_N = 100005;
vector<int> graph[MAX_N];
vector<int> cats(MAX_N, 0);
vector<bool> visited(MAX_N, false);
int maxConsecutiveCats, maxLeaves;

void dfs(int node, int consecutiveCats) {
    visited[node] = true;

    if (cats[node] == 1) {
        consecutiveCats++;
    } else {
        consecutiveCats = 0;
    }

    if (consecutiveCats > maxConsecutiveCats) {
        return;
    }

    bool isLeaf = true;
    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            isLeaf = false;
            dfs(neighbor, consecutiveCats);
        }
    }

    if (isLeaf) {
        maxLeaves++;
    }
```

```
int main() {
    int n, m;
    cin >> n >> m;

    maxConsecutiveCats = m;

    for (int i = 1; i <= n; i++) {
        cin >> cats[i];
    }

    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    maxLeaves = 0;
    dfs(1, 0);

    cout << maxLeaves << endl;

    return 0;
}
```

## Code Explanation

Initialization: The variables and data structures are set up at the beginning of the code. It configures arrays to record visited nodes during traversal and to monitor the existence of cats in each location.

DFS Function: The DFS function is the central component of the solution. It investigates the tree structure, keeping track of how many consecutive cat locations there are, and choosing appropriate eateries.

DFS Traversal: Kefa's house (vertex 1) is the beginning point for the DFS function's tree structure traversal.

Consecutive Cat Locations: When a node without a cat is reached, the count is reset. This allows it to maintain track of the number of consecutive cat locations.

Restaurants That Are Appropriate: The code finds restaurants that are appropriate based on the requirement that the number of consecutive cats not exceed a given threshold.

Leaf Nodes Count: The quantity of leaf nodes—restaurants—that satisfy the requirements is also counted.

Main Function : The programme reads input for the tree structure, processes questions, and outputs the number of restaurants that are appropriate for each query in the main function.

## C - Oil Deposits

This task involves detecting oil deposits in a grid with plots, some containing oil pockets (@) and some empty (*). The objective is to find the number of distinct oil deposits in the grid.

**Code Snippet**

```cpp
#include <iostream>
#include <vector>

using namespace std;


const int dx[] = {1, -1, 0, 0, 1, -1, 1, -1};
const int dy[] = {0, 0, 1, -1, 1, -1, -1, 1};

void dfs(vector<vector<char>>& grid, int x, int y) {
    int m = grid.size();
    int n = grid[0].size();


    grid[x][y] = '*';


    for (int i = 0; i < 8; i++) {
        int newX = x + dx[i];
        int newY = y + dy[i];


        if (newX >= 0 && newX < m && newY >= 0 && newY < n && grid[newX][newY] == '@') {
            dfs(grid, newX, newY);
        }
    }
}

int countOilDeposits(vector<vector<char>>& grid) {
    int m = grid.size();
    int n = grid[0].size();
    int count = 0;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
```

```
            if (grid[i][j] == '@') {
                count++;
                dfs(grid, i, j);
            }
        }
    }

    return count;
}

int main() {
    while (true) {
        int m, n;
        cin >> m >> n;

        if (m == 0) {
            break;
        }

        vector<vector<char>> grid(m, vector<char>(n));

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                cin >> grid[i][j];
            }
        }

        int deposits = countOilDeposits(grid);
        cout << deposits << endl;
    }

    return 0;
}
```

## Code Explanation

Initialization: The variables and data structures are set up at the start of the code. It configures arrays to provide eight potential exploring directions.

DFS Function: The DFS function, which finds oil deposits, is the central component of the system.

Grid Exploration: The method begins at each cell that has an oil pocket (@) and iterates through the grid.

DFS Recursive Calls: The DFS function is called recursively for each oil pocket, exploring neighbouring cells in eight different ways.

Oil Deposit Marking: To denote that visited oil pockets are a part of an oil deposit, they are marked with a '*' by the code.

Counting Deposits: The code keeps track of the number of unique oil deposits as the DFS exploration moves forward.

Main Funtion : The primary purpose of the programme is to keep receiving grid input, process each grid, and output the quantity of unique oil deposits found in each grid.

D - Travelling cost

In this task, the government of Spoj_land has constructed roads between various locations, each with a fixed cost. Rohit wants to find the minimum cost for traveling from a source location to other destinations. The problem involves performing multiple queries to find the minimum cost.

## Code Snippet

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <map>
#include <climits>

using namespace std;

const int MAX_LOCATIONS = 501;
const int INF = INT_MAX;

vector<pair<int, int>> graph[MAX_LOCATIONS];
map<pair<int, int>, int> costs;

int dijkstra(int start, int target) {
    vector<int> distance(MAX_LOCATIONS, INF);
    distance[start] = 0;

    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
    pq.push({0, start});

    while (!pq.empty()) {
        int dist = pq.top().first;
        int node = pq.top().second;
        pq.pop();

        if (node == target) {
            return dist;
        }

        if (dist > distance[node]) {
            continue;
        }

        for (const auto& edge : graph[node]) {
            int to = edge.first;
            int weight = edge.second;
```

```
            if (dist + weight < distance[to]) {
                distance[to] = dist + weight;
                pq.push({distance[to], to});
            }
        }
    }
    return INF;
}
int main() {
    int N;
    cin >> N;
    for (int i = 0; i < N; i++) {
        int A, B, W;
        cin >> A >> B >> W;
        graph[A].push_back({B, W});
        graph[B].push_back({A, W});
        costs[{A, B}] = W;
        costs[{B, A}] = W;
    }
    int U;
    cin >> U;
    int Q;
    cin >> Q;
    while (Q--) {
        int V;
        cin >> V;
        int minCost = dijkstra(U, V);

        if (minCost == INF) {
            cout << "NO PATH" << endl;
        } else {
            cout << minCost << endl;
        }
    }
    return 0;
}
```

**Code Explanation**

Initialization: Variables, data structures, and constants are initialised by the code. It specifies a maximum number of nodes as well as an infinity constant.

Adjacency List: An adjacency list, abbreviated as adj, is a vector of pairs, each of which represents an edge. The destination node is indicated by the first element of the pair, while the edge's weight or cost is shown by the second element.

Dijkstra's Function: Using a priority queue, the dijkstra function determines the shortest paths between each given source node and every other node in the graph. This is the fundamental component of the solution.

Main Function: To determine the lowest cost of transportation from the source node to each query destination, the programme calls the dijkstra function after reading input for the roads built and initialising the distance array for shortest distances.

Handling Query: The programme answers queries by determining whether a path connects the source and destination nodes. If it does, it writes "NO PATH" or the minimal cost.